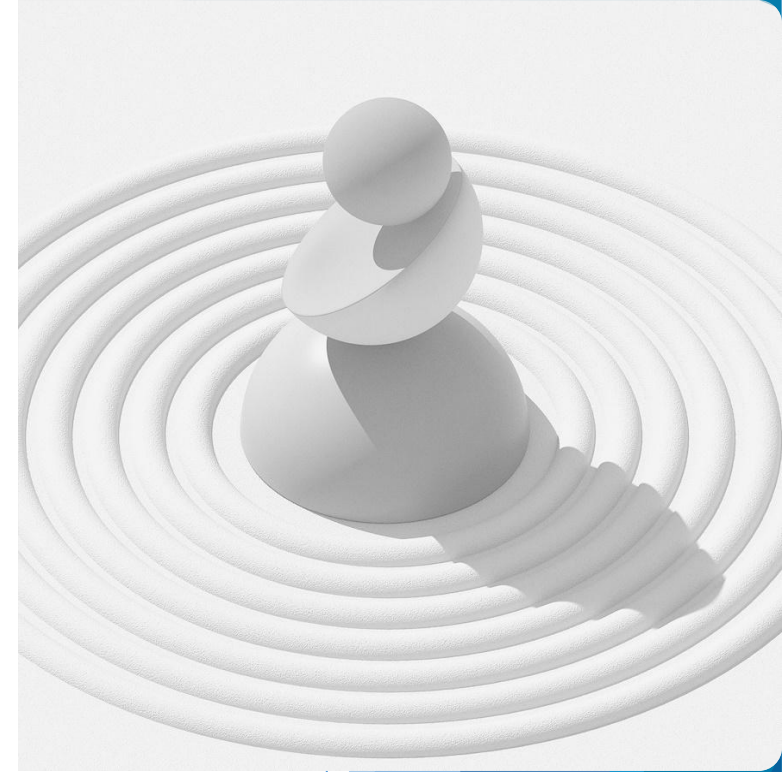


Presented by Nirvana Popescu

Lesson 2: Software Development Life Cycle (SDLC)

Outline

- ▶ Definition of SDLC
- ▶ Why structured lifecycles are necessary
- ▶ SDLC phases
- ▶ SDLC models (Waterfall, Agile/Iterative , V-Cycle)
- ▶ Traceability in software engineering
- ▶ Integrating **Scade One** into the SDLC



What is SDLC?

- ▶ Definition of Software Development Life Cycle
 - ❑ A structured process used to develop software systems
 - ❑ Defines phases, activities, and deliverables
 - ❑ Ensures software is developed systematically and efficiently
 - ❑ Used widely in complex and safety-critical systems
- ▶ Goal: deliver reliable, maintainable, and high-quality software.

Why Structured Lifecycles Are Needed?

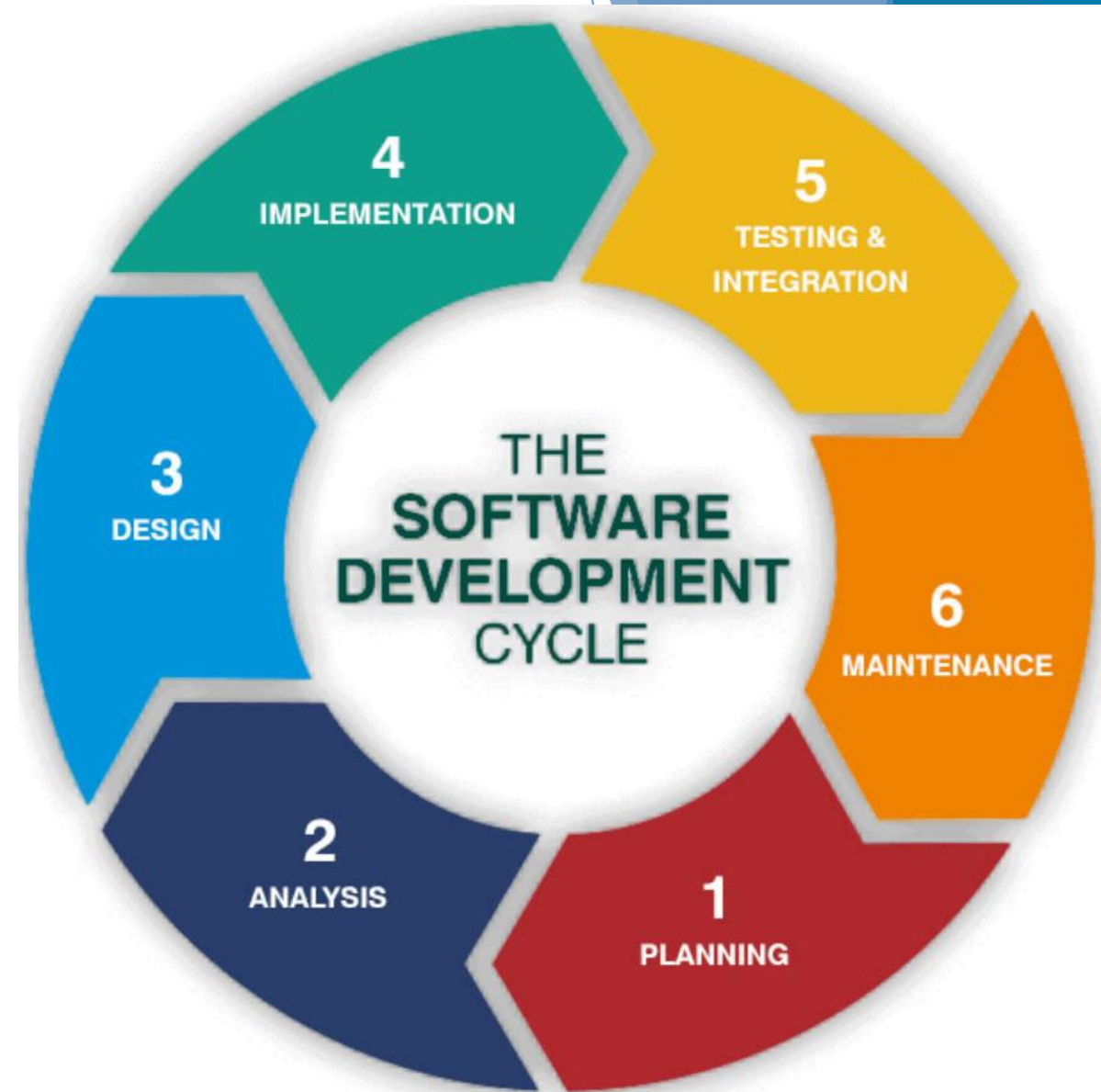
- ▶ Software projects are complex and require coordination.
- ▶ Structured lifecycles help with:
 - ❑ Planning development activities
 - ❑ Managing complexity
 - ❑ Reducing risks and errors
 - ❑ Ensuring quality and safety
 - ❑ Tracking progress and documentation
 - ❑ Improving communication between stakeholders
- ▶ Especially important in safety-critical domains.

Typical SDLC Phases

The SDLC typically includes the following stages:

- ▶ Requirements Analysis
- ▶ System and Software Design
- ▶ Implementation (Coding)
- ▶ Verification and Validation
- ▶ Maintenance

These phases guide the software from initial idea to operational system.



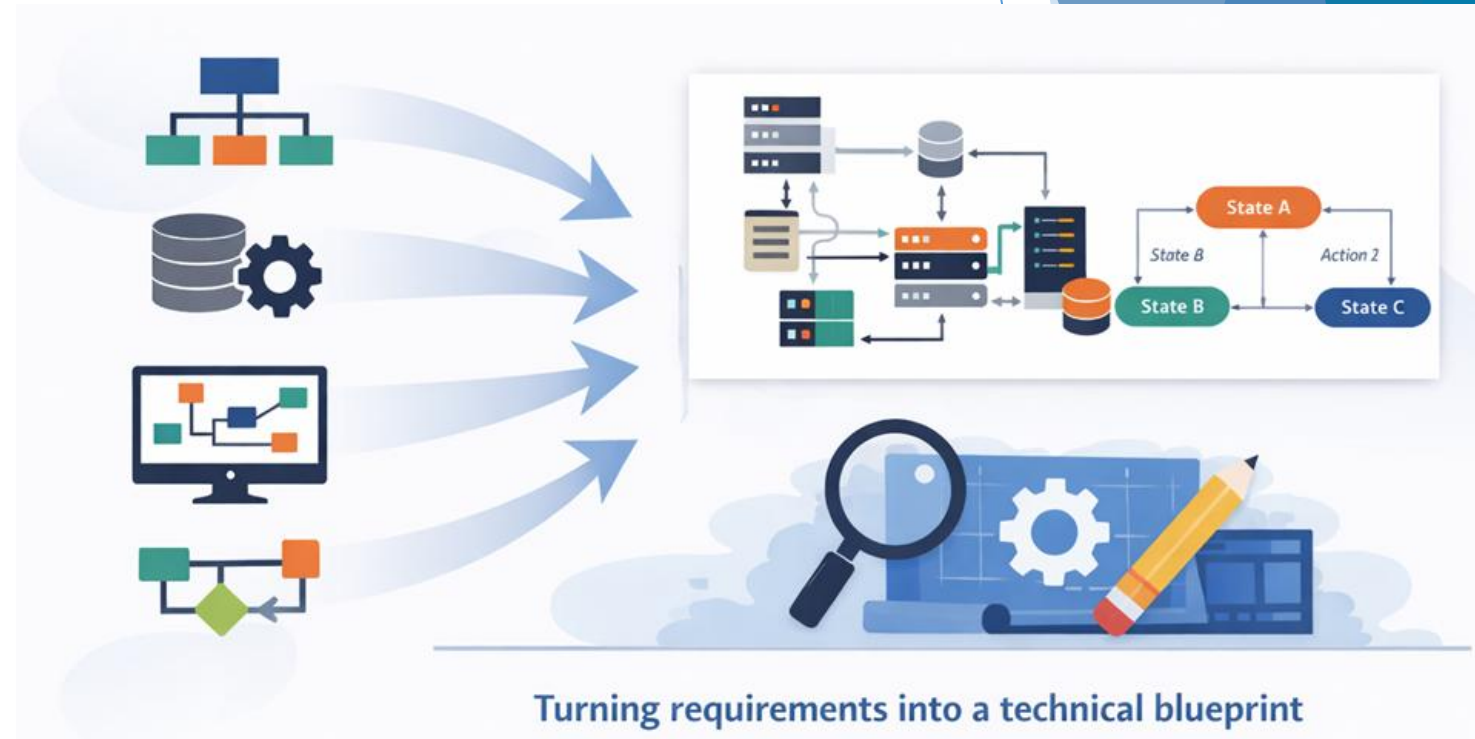
Requirements Analysis

- ▶ Goal: understand what the system must do.
- ▶ Activities include:
 - ❑ Gathering stakeholder requirements
 - ❑ Defining functional requirements
 - ❑ Defining non-functional requirements
 - ❑ Creating requirement specifications
- ▶ Output:
 - ❑ Software Requirements Specification (SRS)
- ▶ Clear requirements reduce misunderstandings later.



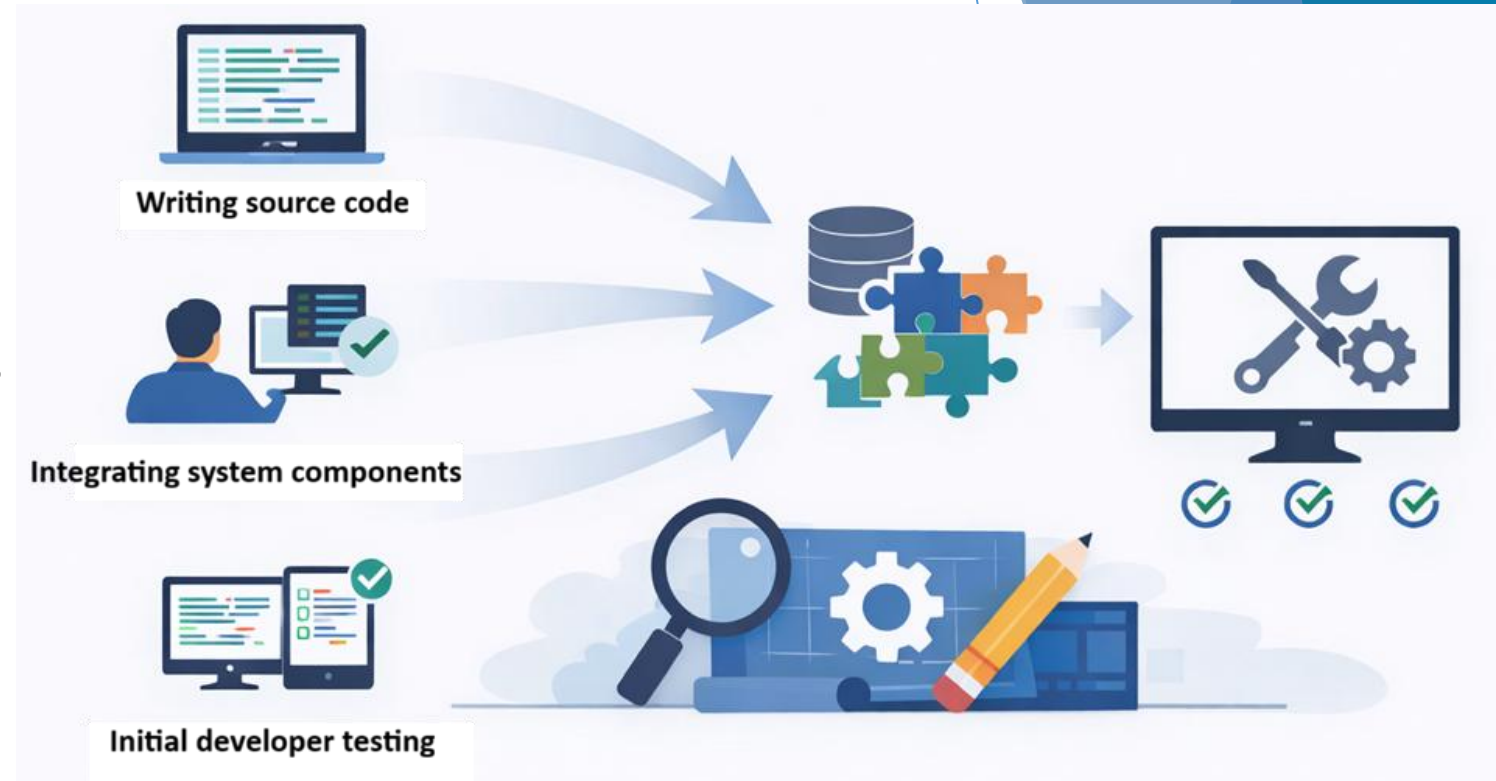
Design Phase

- ▶ The design phase defines how the system will work.
- ▶ Key activities:
 - ❑ System architecture design
 - ❑ Data and component design
 - ❑ Interface design
 - ❑ Modeling system behavior
- ▶ Outputs:
 - ❑ Architecture diagrams
 - ❑ Design models
- ▶ Design translates requirements into a technical solution.



Implementation Phase

- ▶ Implementation is where the system is built.
- ▶ Activities include:
 - ❑ Writing source code
 - ❑ Integrating system components
 - ❑ Initial developer testing
- ▶ Goals:
 - ❑ Implement the design accurately
 - ❑ Maintain code quality and consistency



Verification & Validation

- ▶ Ensures the system is correct and meets requirements.
- ▶ Verification:
"Are we building the system right?"
- ▶ Validation: "Are we building the right system?"
- ▶ Typical activities:
 - ❑ Unit testing
 - ❑ Integration testing
 - ❑ System testing
 - ❑ Acceptance testing



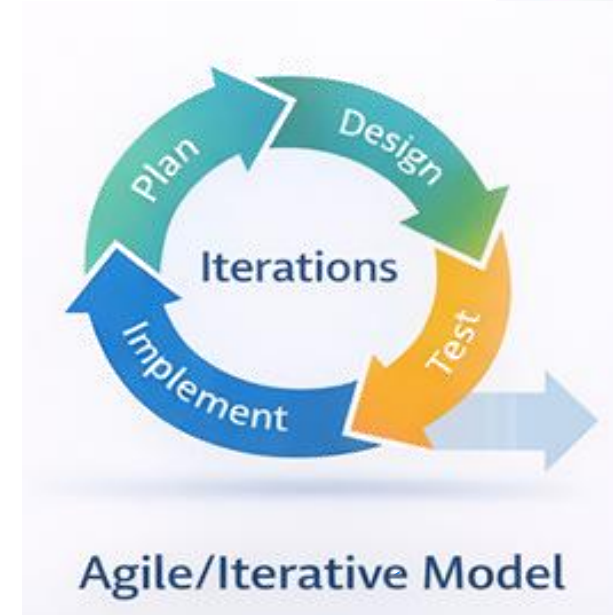
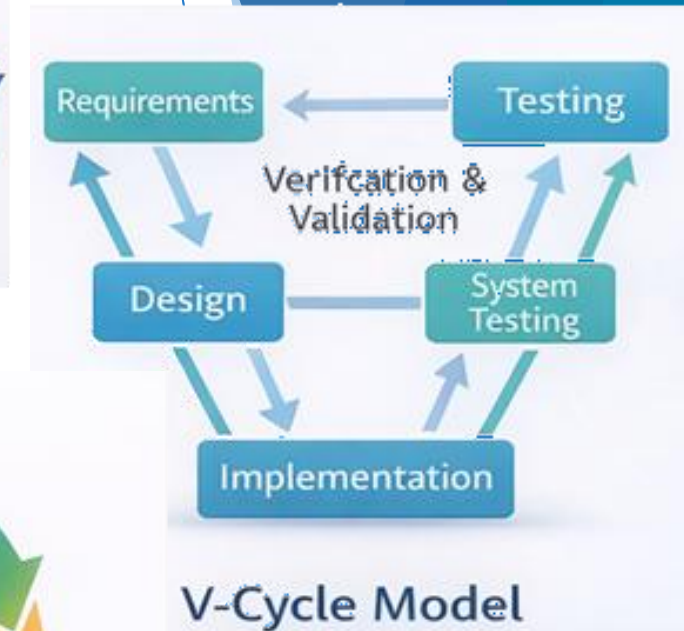
Maintenance Phase

- ▶ After deployment, software must be maintained.
- ▶ Types of maintenance:
 - ❑ Corrective: fixing defects
 - ❑ Adaptive: adapting to new environments
 - ❑ Perfective: improving performance
 - ❑ Preventive: improving maintainability
- ▶ Maintenance can represent a large part of the software lifecycle.



SDLC Models

- ▶ Different models organize SDLC phases differently.
- ▶ Common models include:
 - ▶ Waterfall Model
 - ▶ Agile/Iterative Model
 - ▶ V-Cycle Model
- ▶ Each model has different strengths depending on project type and requirements.



Differences Between SDLC Models

Waterfall Model - Traditional Sequential Approach

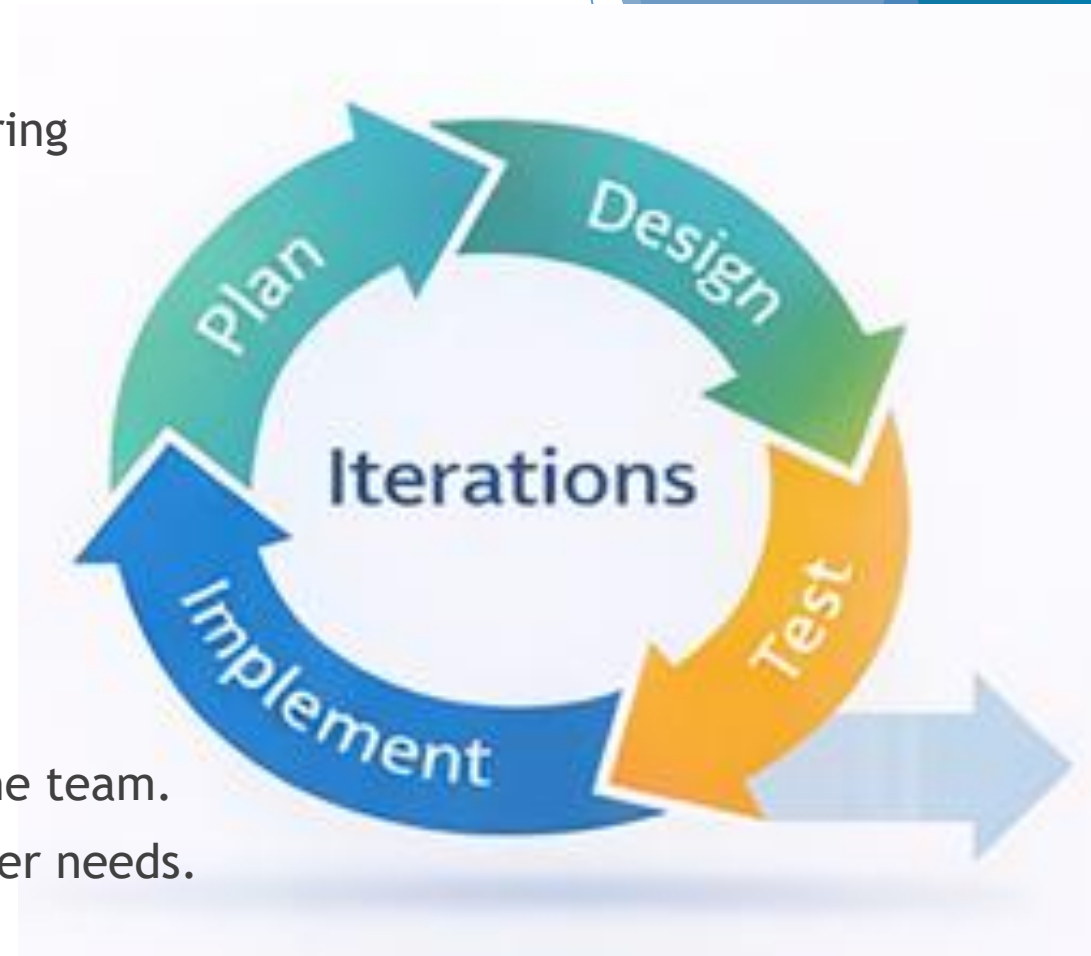
- ▶ Process: Linear, sequential phases: Requirements → Design → Implementation → Testing → Deployment → Maintenance.
- ▶ Key Features:
 - ▶ Each phase must be completed before the next begins.
 - ▶ Strong documentation is produced at each phase, which acts as a reference.
 - ▶ Changes are difficult and costly once a phase is finished.
- ▶ Advantages:
 - ▶ Clear structure and milestones.
 - ▶ Easy to manage and measure progress.
 - ▶ Suitable for projects with stable and well-defined requirements.
- ▶ Limitations:
 - ▶ Poor flexibility for changing requirements.
 - ▶ Late testing may reveal significant issues.



Agile Model (Iterative & Incremental)

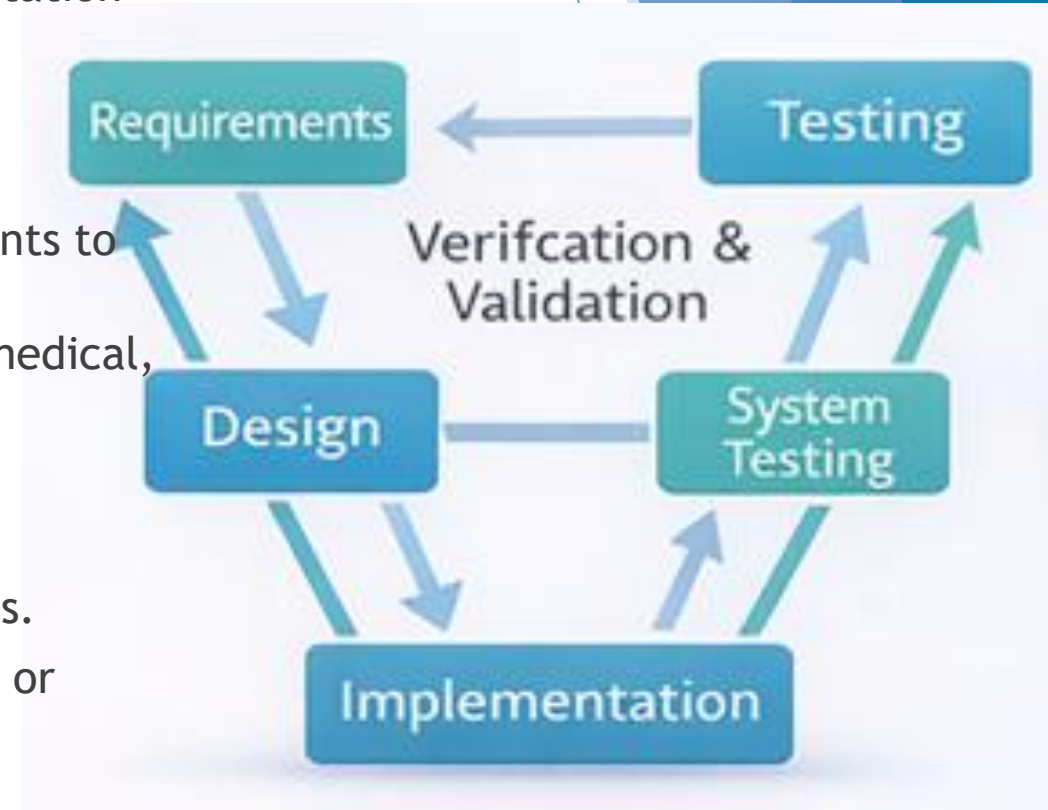
- Flexible and Iterative

- ▶ Process: Development is iterative and incremental, delivering working software in short cycles (sprints).
- ▶ Key Features:
 - ▶ Continuous feedback from users and stakeholders.
 - ▶ Frequent releases allow early detection of issues.
 - ▶ Requirements are flexible and can evolve throughout development.
- ▶ Advantages:
 - ▶ Adaptable to changing requirements.
 - ▶ Encourages collaboration and communication within the team.
 - ▶ Reduces risk of building software that doesn't meet user needs.
- ▶ Limitations:
 - ▶ Less emphasis on detailed documentation.
 - ▶ May be harder to manage for very large or safety-critical systems.



V-Model (Verification & Validation)

- ▶ Process: Each development phase has a corresponding test phase, forming a “V” shape:
 - ▶ Requirements → System Design → Architecture → Implementation → Testing (Unit → Integration → System → Acceptance)
- ▶ Key Features:
 - ▶ Strong focus on verification and validation.
 - ▶ High-quality documentation and traceability from requirements to testing.
 - ▶ Commonly used in safety-critical systems (e.g., aerospace, medical, automotive).
- ▶ Advantages:
 - ▶ Systematic and disciplined approach.
 - ▶ Early identification of defects through planned testing phases.
 - ▶ Ideal for projects where failure can cause injury, loss of life, or major damage.
- ▶ Limitations:
 - ▶ Less flexible to changing requirements compared to Agile.
 - ▶ Can be time-consuming and more costly.



Why Safety-Critical Systems Prefer Iterative / V-Cycle Models Over Pure Agile

- ▶ Safety-critical systems (aerospace, automotive, railway, medical) require strict engineering processes because failures can lead to injury, loss of life, or major damage.

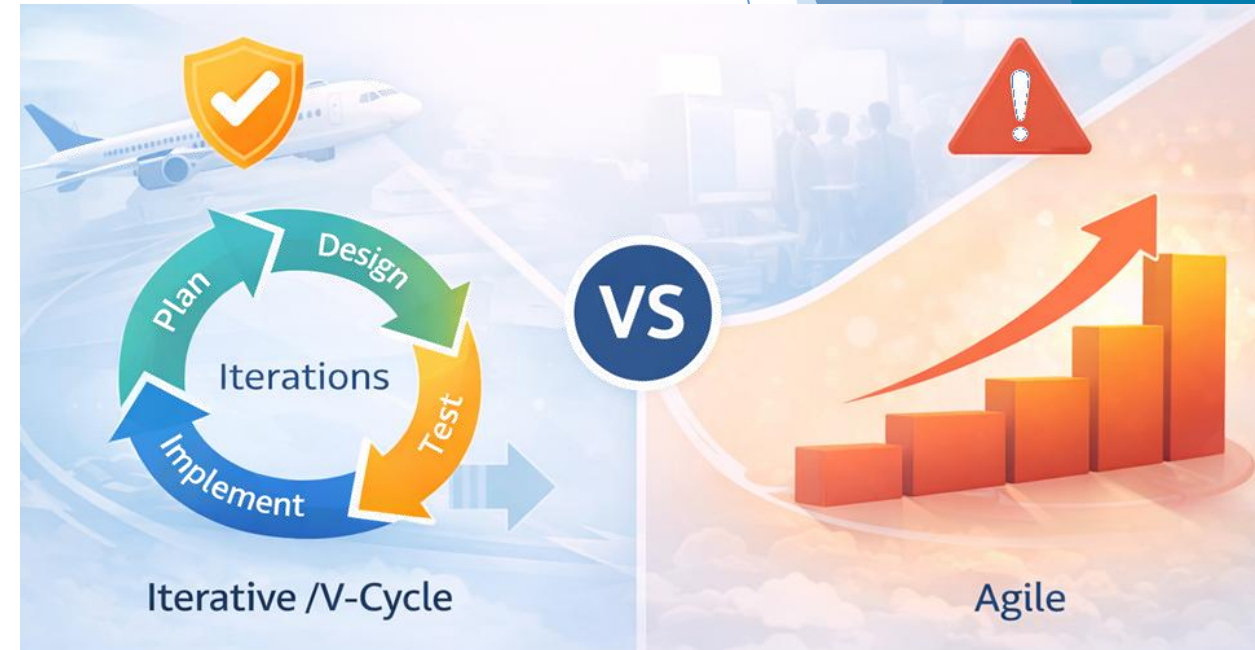
- ▶ Key Reasons

1. Strong Traceability

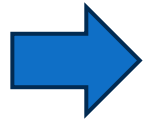
- ❑ Every requirement must be traceable to design, code, and tests
- ❑ Required for certification and safety audits

2. Rigorous Verification & Validation

- ❑ Each development phase is paired with a testing phase
- ❑ Ensures the system meets all safety requirements



Why Safety-Critical Systems Prefer Iterative / V-Cycle Models Over Pure Agile



3. Compliance with Safety Standards

Examples:

- ❑ DO-178C (aerospace)
- ❑ ISO 26262 (automotive)
- ❑ EN 50128 (railway)

These standards require documentation and verification evidence.

4 Predictability and Risk Control

- ❑ Structured lifecycles support risk analysis and deterministic behavior
- ❑ Reduces unexpected system behavior



Important clarification

Agile can be used in safety-critical systems, but usually not alone.

Many organizations adopt hybrid approaches, combining Agile practices with structured models like the V-Cycle.

This approach maintains flexibility while ensuring strict verification, documentation, and traceability.

Because safety is critical, structured and well-controlled development processes are still essential.

Integrating Scade One into the SDLC

- ▶ Scade One supports multiple SDLC phases.
- ▶ It is mainly used in:
 - ▶ Requirements analysis
 - ▶ Design
 - ▶ Implementation
 - ▶ Verification & validation
- ▶ This integration improves development efficiency and safety.

Phase	Traditional Impact	Scade One Impact
Design	Manual diagrams/docs	Executable behavioral models
Implementation	Manual coding (error-prone)	Automatic, certified code generation
V&V	Late-stage bug discovery	Early simulation and formal proof

Benefits of Using Scade One

► Key advantages:

- ❑ Early validation
 - Detect design issues early
 - ❑ Deterministic behavior
 - Predictable system execution
 - ❑ Traceability
 - Links between requirements, design, and code
 - ❑ Certified code generation
 - Generates reliable and certifiable code for safety-critical systems
- These features make Scade One suitable for aerospace, automotive, and rail systems.

Summary

- ▶ **Understanding SDLC:** what the Software Development Life Cycle is and why structured processes are necessary for managing complex software systems.
- ▶ **SDLC Phases:** exploration of the main stages, from requirements analysis to maintenance.
- ▶ **Development Models:** comparison between Waterfall, Agile, and V-Cycle, highlighting that iterative models are often preferred in safety-critical systems due to stronger control, documentation, and verification.
- ▶ **Traceability and Tools:** importance of traceability and verification, and how **Scade One** supports several SDLC phases through model-based development, early validation, and certified code generation.

